NIT2112
Object Oriented Programming

Session 05

Design Patterns

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Acknowledgement of Country

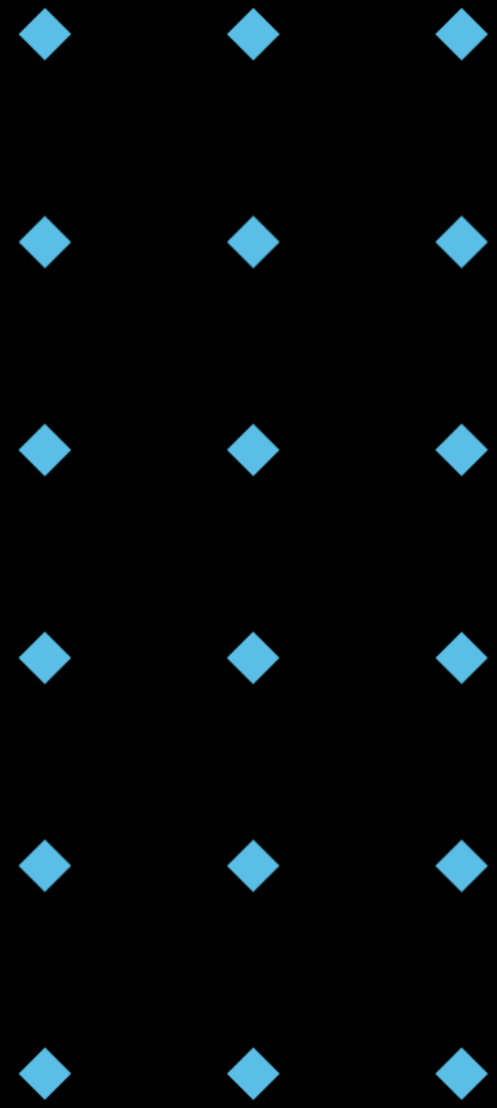
Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- Design Patterns: Concept & Importance
- Creational Patterns (Singleton, Factory Method, Builder)
- Structural Patterns (Adapter, Decorator)
- Behavioural Patterns (Observer)
- Guidance on Choosing Design Patterns

Discussion Time!

- Question: Can you think of a situation in your previous programming experience where you encountered a recurring design problem? How did you solve it?



What are Design Patterns?

- As systems grow, maintaining clarity and efficiency becomes a challenge.
- Design patterns help mitigate these issues

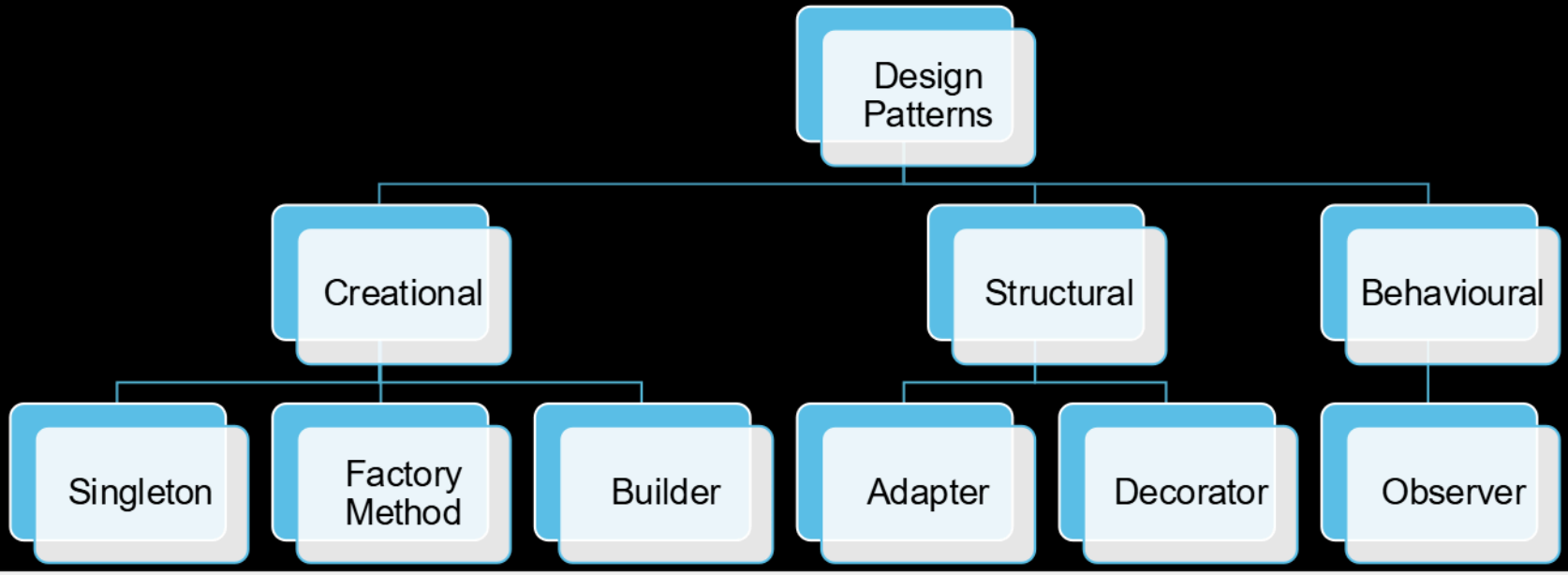
Design Patterns

- **Reusable solutions** to common **software design** problems.
- Templates for structuring code to enhance **maintainability** and **scalability**.
- Promote **code reuse**.
- Independent of specific programming languages – a **conceptual blueprint**.
- Promote **best practices** for object-oriented design.
- Enhance team communication through a shared vocabulary.

Categories of Design Patterns

- **Creational**: Focus on object **creation** mechanisms.
- **Structural**: Deal with the **composition** of classes and objects.
- **Behavioural**: Concerned with **communication** between objects.

- There are 23 recognized design patterns. But we will only cover a selection of 6.



Creational Design Patterns

- Abstract the object instantiation process.
- Decouple a system from the specifics of object creation.
- Examples: Singleton, Factory, Builder.

Creational Patterns: Singleton

- Ensures a class has only one instance.
- Provides a global point of access to it.
- Useful for managing resources like database connections or configuration settings.

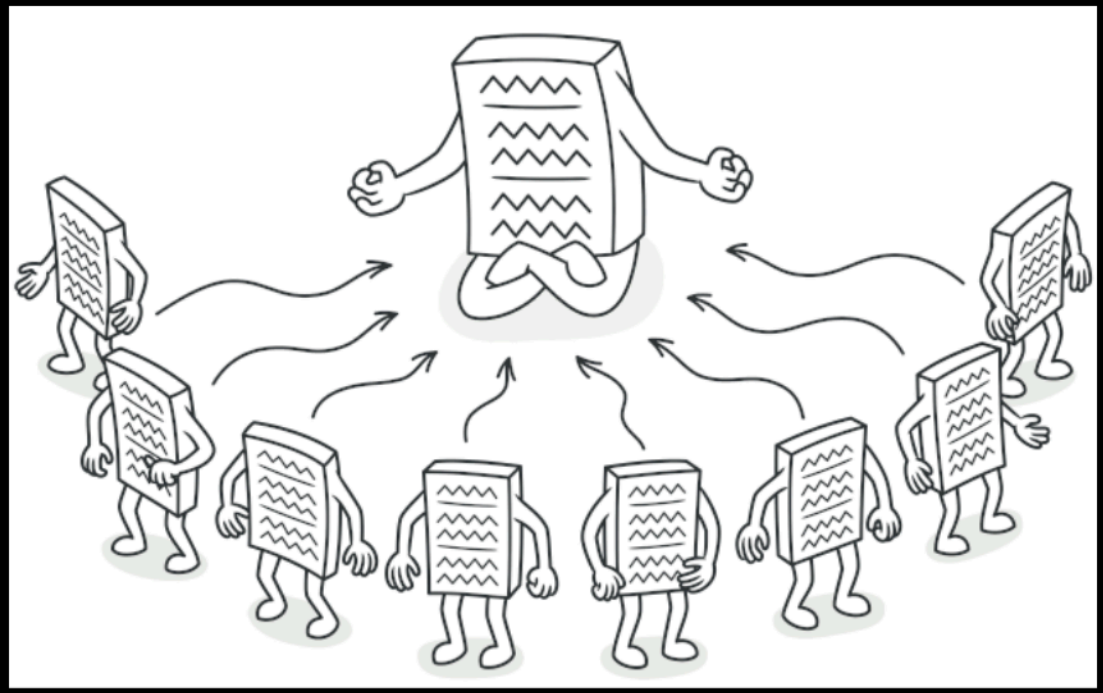


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/singleton

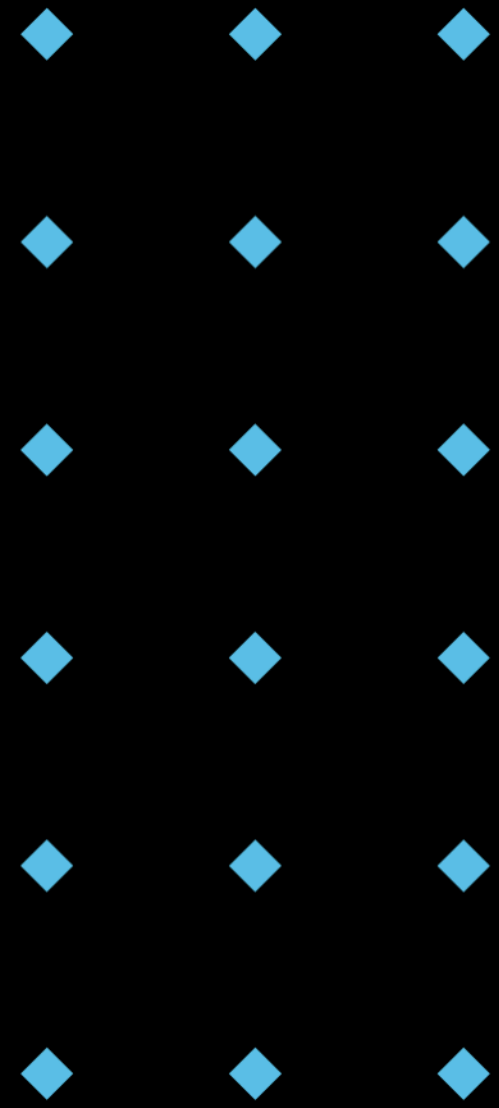
Singleton - Example

- *GameManagerMeta* is a metaclass that controls class creation.
- *_instances* dictionary stores the single instance.
- *__call__* method ensures only one instance is created.
- *id(game_manager1) == id(game_manager2)* confirms they are the same object.

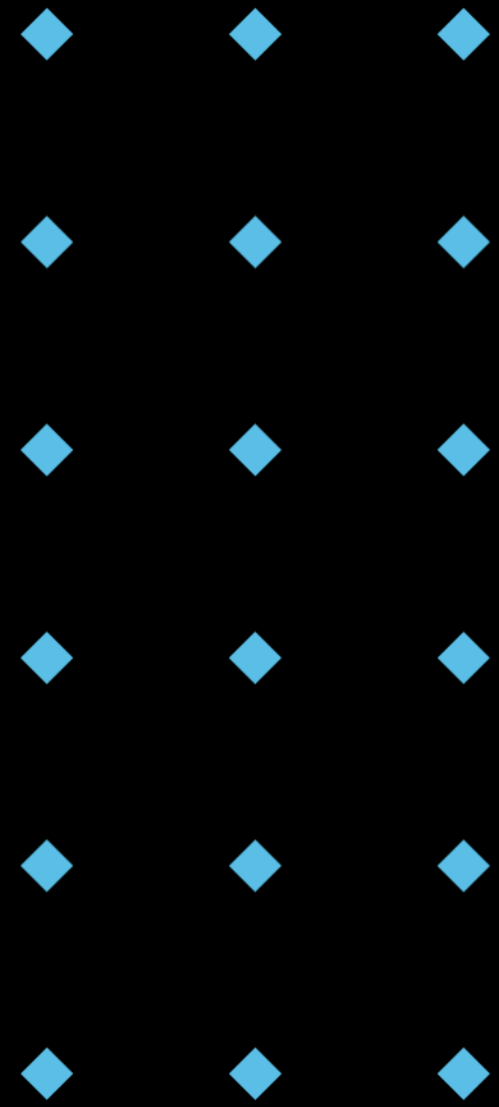
```python
class GameManagerMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            instance = super().__call__(*args, **kwargs)
            cls._instances[cls] = instance
        return cls._instances[cls]


class GameManager(metaclass=GameManagerMeta):
    def __init__(self):
        self.score = 0

    def add_score(self, points):
        self.score += points
        print(f"Score updated: {self.score}")


# Usage
game_manager1 = GameManager()
game_manager2 = GameManager()

game_manager1.add_score(100)  # Output: Score updated: 100
game_manager2.add_score(50)   # Output: Score updated: 150 (Why?)

assert id(game_manager1) == id(game_manager2)
```

Singleton - Use Case

- Logging
- Configuration Management
- Caching
- Thread Pools



Singleton: Benefits & Drawbacks



Benefits

- Controlled access to a shared resource.
- Saves memory by creating only one instance.

Drawbacks

- Can reduce testability.
- Can hide dependencies.

Creational Patterns: Factory Method

- Defines an interface for creating objects.
- Let subclasses alter the type of objects that will be created.
- Promotes loose coupling by reducing dependencies.

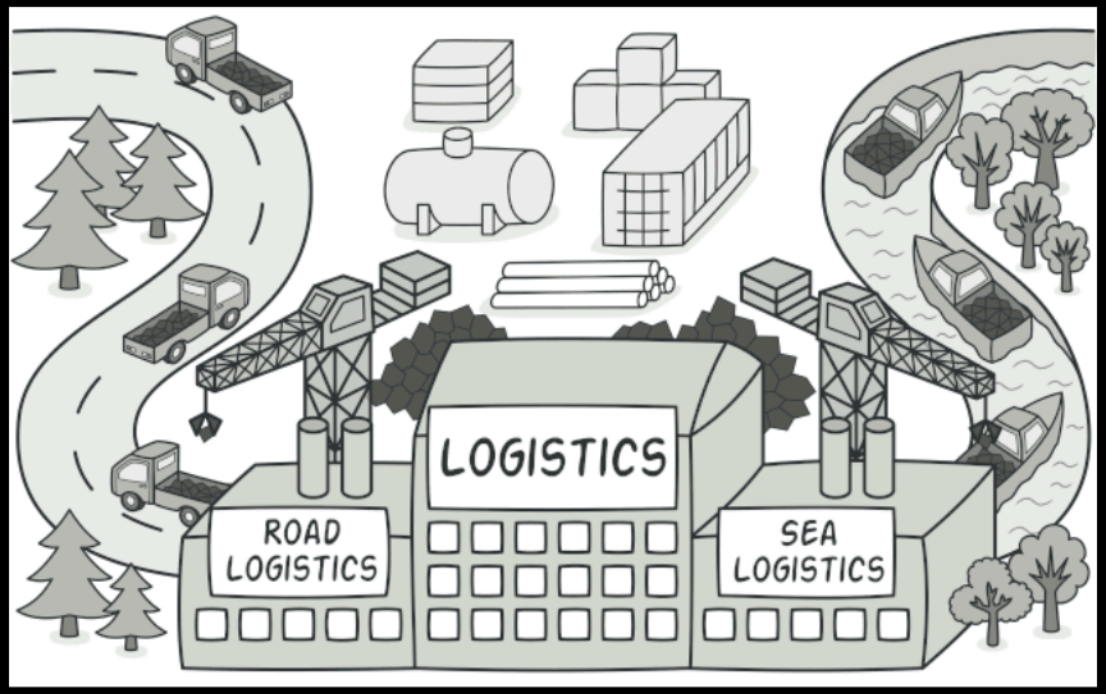


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/factory-method

Factory Method - Example

- Problem: We need to create different types of pizzas (Pepperoni, Veggie) based on customer order.
- Why Not Just if/else? Imagine adding 10 more pizza types! The if/else chain would become huge and hard to manage in the main code.
- Solution: Factory Method! The PizzaFactory class encapsulates the pizza creation logic. It knows how to create each type of pizza.

```python
from abc import ABC, abstractmethod

# Product: Pizza
class Pizza(ABC):
    @abstractmethod
    def prepare(self) -> str:
        pass

# Concrete Products: Specific Pizza types
class PepperoniPizza(Pizza):
    def prepare(self) -> str:
        return "Preparing Pepperoni!"

class VeggiePizza(Pizza):
    def prepare(self) -> str:
        return "Preparing Veggie Pizza!"

# Creator: Pizza Factory
class PizzaFactory:
    def create_pizza(self, pizza_type):
        if pizza_type == "pepperoni":
            return PepperoniPizza()
        elif pizza_type == "veggie":
            return VeggiePizza()
        else:
            raise ValueError("Invalid pizza type")

# Client code:
factory = PizzaFactory()

pizza1 = factory.create_pizza("pepperoni")
print(pizza1.prepare())

pizza2 = factory.create_pizza("veggie")
print(pizza2.prepare())

#pizza3 = factory.create_pizza("sausage") # This would cause an error
```

Factory Method - Use Case

- When a class cannot anticipate the class of objects it must create.
- When the system wants to localize the knowledge of which class gets instantiated.

Factory Method: Benefits & Drawbacks

Benefits

- Loose Coupling: Decouples client code from concrete product classes.
- Extensibility: Easy to add new product types without modifying existing code.
- Single Responsibility: Each class has a clear responsibility.

Drawbacks

- Increased Complexity: Introduces more classes and interfaces, potentially increasing code complexity.
- Abstract Creator: Can be challenging to implement if the product creation logic is complex and varies significantly.

Creational Patterns: Builder

- Separates the construction of a complex object from its representation.
- Allows the same construction process to create different representations.
- Useful for constructing complex objects incrementally.

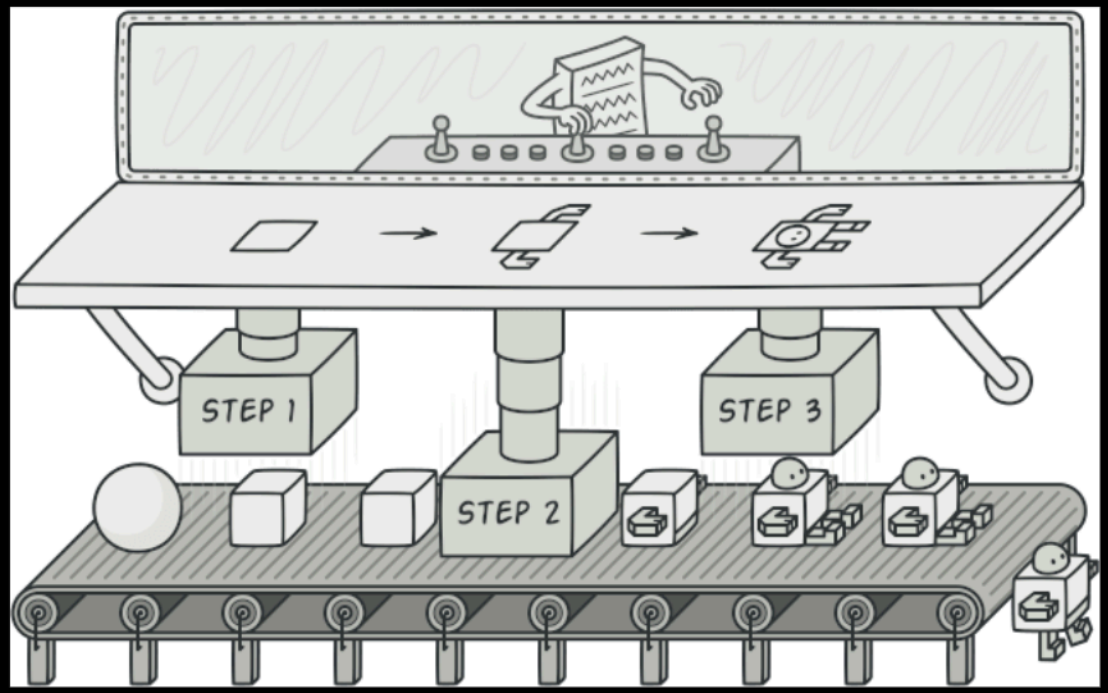


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/builder

Builder - Example

```python
class Computer:
    def __init__(self):
        self.ram = None
        self.cpu = None
        self.storage = None

    def display(self):
        print("Computer Specs:")
        print(f"  RAM: {self.ram}")
        print(f"  CPU: {self.cpu}")
        print(f"  Storage: {self.storage}")


class ComputerBuilder:
    def __init__(self):
        self.computer = Computer()

    def set_ram(self, ram):
        self.computer.ram = ram
        return self

    def set_cpu(self, cpu):
        self.computer.cpu = cpu
        return self

    def set_storage(self, storage):
        self.computer.storage = storage
        return self

    def build(self):
        return self.computer
```

```python
# Client Code
computer1 = ComputerBuilder() \
    .set_ram("16GB") \
    .set_cpu("Intel i5") \
    .set_storage("512GB SSD") \
    .build()

computer2 = ComputerBuilder() \
    .set_ram("32GB") \
    .set_cpu("AMD Ryzen 7") \
    .set_storage("1TB NVMe") \
    .build()

print("Computer 1:")
computer1.display()

print("\nComputer 2:")
computer2.display()
```

Builder - Use Case

- Requires creation of complex entities with numerous optional parameters and settings.
- Encapsulates the construction logic within a builder, providing control over the configuration and construction process.

Builder: Benefits & Drawbacks

- **Benefits**
- Clean and Readable: Client code is more readable and easier to understand.
- Simplified Object Creation: The builder handles the complex logic of object creation.
- Flexibility: Can create different representations of the same object using the same construction process.

- **Drawbacks**
- More Code: The builder pattern requires creating separate builder classes.
- Changing the product after building: Changes to the class is hard to implement.

Discussion Time!

- Question: How does the Builder pattern differ from the Factory Method pattern? In what scenarios would you choose one over the other?

Key Differences between Factory Method and Builder

- Use **Factory Method** to create different kinds of things.
- Use **Builder** to create one kind of thing in different ways.

	Factory Method	Builder
Intent	Defines an interface for creating an object, but lets subclasses decide which class to instantiate. It defers instantiation to subclasses. Focuses on creating different types of objects.	Separates the construction of a complex object from its representation so that the same construction process can create different representations. Focuses on constructing a complex object step-by-step.
Complexity of Object	Typically used when the object creation is relatively simple. The focus is on which object to create, not how to create it.	Used when the object creation is complex, involving multiple steps, optional components, and varying representations.
Process vs. Product	Concerned with the creation process itself – how the object is instantiated.	Concerned with the step-by-step construction of the object and the final product that results.

Structural Design Patterns

- Simplify design by identifying relationships between entities.
- Deal with the composition of classes or objects.
- Examples: Adapter, Decorator, Composite.

Structural Patterns: Adapter

- Converts the interface of a class into another interface clients expect.
- Allows classes with incompatible interfaces to work together.
- Acts as a connector between two disparate types.

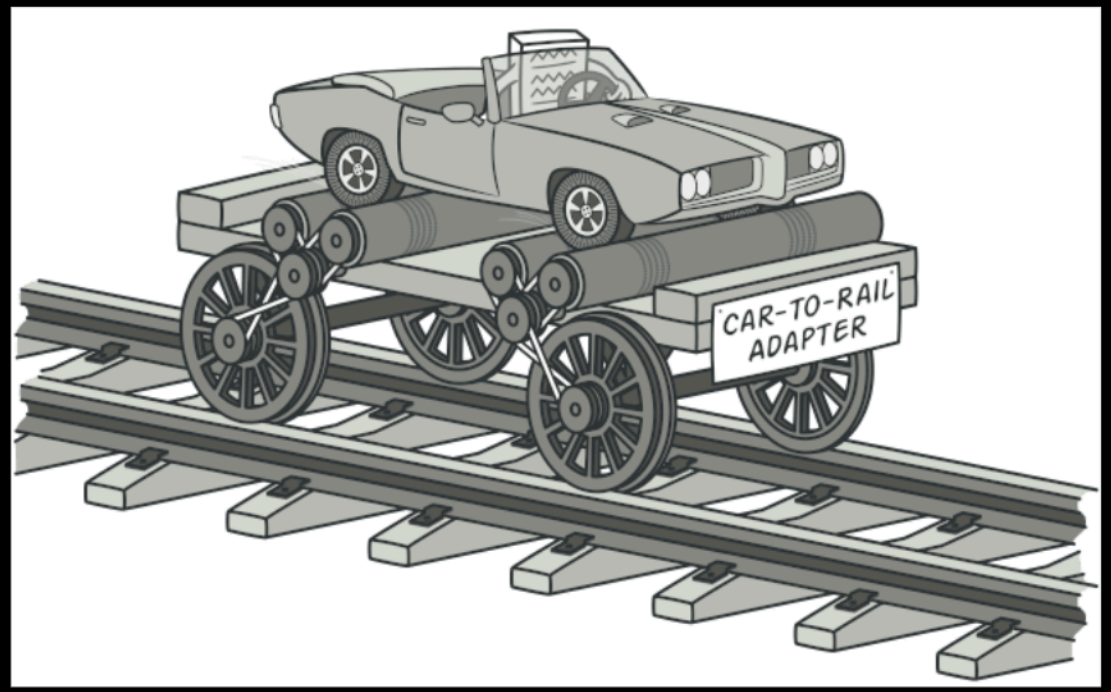


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/adapter

Adapter - Example

```python
class CelsiusTemperature:
    def get_temperature(self) -> float:
        return 25.0  # Default temperature in Celsius

class FahrenheitTemperatureSensor:
    def read_temperature(self) -> float:
        return 77.0  # Default temperature in Fahrenheit

class TemperatureAdapter:
    def __init__(self, fahrenheit_sensor: FahrenheitTemperatureSensor):
        self.fahrenheit_sensor = fahrenheit_sensor

    def get_temperature(self) -> float:
        fahrenheit = self.fahrenheit_sensor.read_temperature()
        celsius = (fahrenheit - 32) * 5/9
        return celsius

# Client Code
celsius_thermometer = CelsiusTemperature()
print(f"Temperature in Celsius: {celsius_thermometer.get_temperature()}")

fahrenheit_sensor = FahrenheitTemperatureSensor()
adapter = TemperatureAdapter(fahrenheit_sensor)
print(f"Temperature from Fahrenheit Sensor (converted to Celsius): {adapter.get_temperature()}")
```

Adapter - Use Case

- Useful in scenarios where we have a legacy interface that cannot be changed.
- Integrate a legacy interface with a newer system interface.

Structural Patterns: Decorator

- Adds behaviour to individual objects dynamically.
- Without affecting the behaviour of other objects from the same class.
- Key to extending functionalities in a flexible manner.

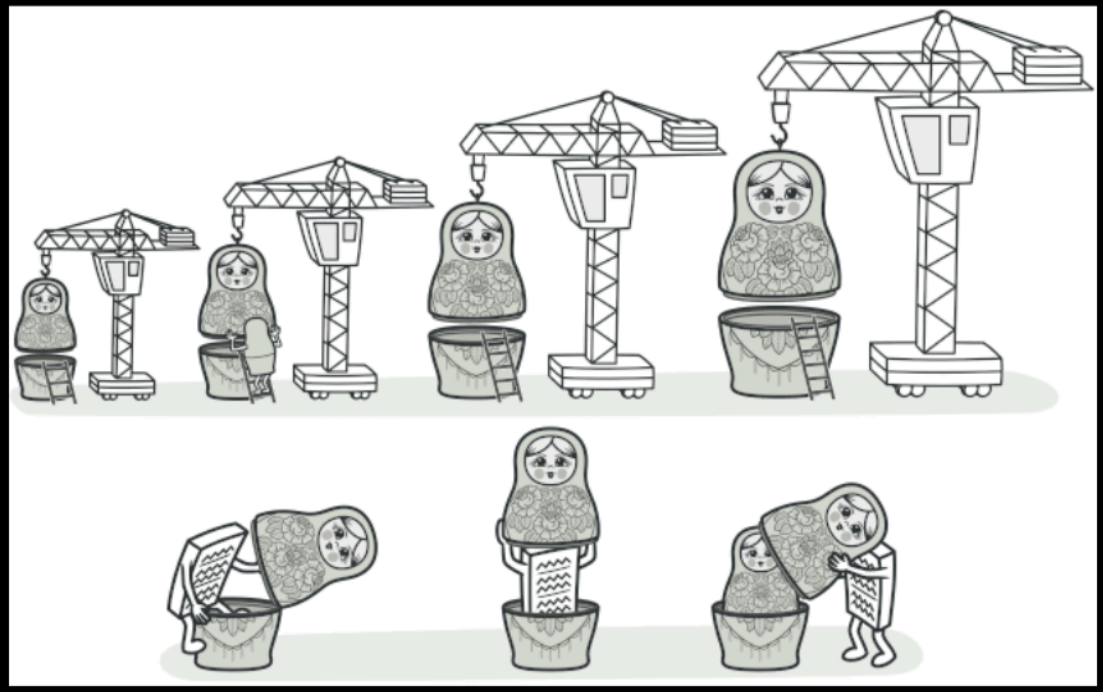


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/decorator

Decorator - Example

```python
class Coffee:
    def get_cost(self) -> float:
        return 2.0

    def get_description(self) -> str:
        return "Simple coffee"

class MilkDecorator:
    def __init__(self, coffee: Coffee):
        self.coffee = coffee

    def get_cost(self) -> float:
        return self.coffee.get_cost() + 0.5

    def get_description(self) -> str:
        return self.coffee.get_description() + ", with milk"

class SugarDecorator:
    def __init__(self, coffee: Coffee):
        self.coffee = coffee

    def get_cost(self) -> float:
        return self.coffee.get_cost() + 0.2

    def get_description(self) -> str:
        return self.coffee.get_description() + ", with sugar"
```

```python
# Client Code
my_coffee = Coffee()
print(f"{my_coffee.get_description()} costs \
    ${my_coffee.get_cost()}")
# Output: Simple coffee costs     $2.0

my_coffee_with_milk = MilkDecorator(my_coffee)
print(f"{my_coffee_with_milk.get_description()} costs \
    ${my_coffee_with_milk.get_cost()}")
# Output: Simple coffee, with milk costs     $2.5

my_coffee_with_milk_and_sugar = SugarDecorator(my_coffee_with_milk)
print(f"{my_coffee_with_milk_and_sugar.get_description()} costs \
    ${my_coffee_with_milk_and_sugar.get_cost()}")
# Output: Simple coffee, with milk, with sugar costs     $2.7
```

Decorator - Use Case

- Commonly used for implementing features like logging, access control, and input validation.
- Extending functionalities in a flexible manner.

Behavioural Design Patterns

- Concerned with communication between objects.
- Facilitate methods to coordinate complex flows of control and data.
- Examples: Observer, Strategy, Command.

Behavioural Patterns: Observer

- Implements distributed event-handling systems.
- Multiple objects need to observe and react to changes in another object.
- Decoupling the sender from receivers.

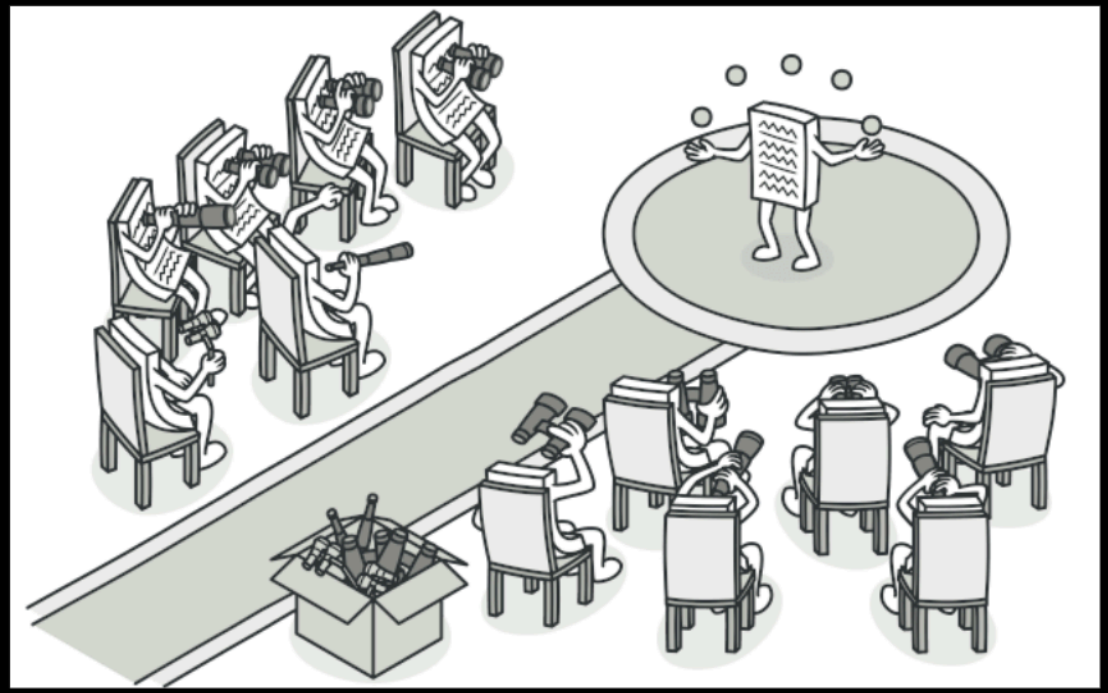


Image from Refactoring.Guru., 2025, https://refactoring.guru/design-patterns/observer

Observer - Example

```python
from abc import ABC, abstractmethod
from typing import List

class WeatherStation:
    def __init__(self):
        self._observers: List['WeatherObserver'] = []
        self._temperature: float = 0

    def attach(self, observer: 'WeatherObserver') -> None:
        self._observers.append(observer)

    def detach(self, observer: 'WeatherObserver') -> None:
        self._observers.remove(observer)

    def notify(self) -> None:
        for observer in self._observers:
            observer.update(self._temperature)

    def set_temperature(self, temperature: float) -> None:
        self._temperature = temperature
        print(f"Weather Station: Temperature changed to {self._temperature}")
        self.notify()

class WeatherObserver(ABC):
    @abstractmethod
    def update(self, temperature: float) -> None:
        pass

class PhoneDisplay(WeatherObserver):
    def update(self, temperature: float) -> None:
        print(f"Phone Display: Temperature is now {temperature}")

class SmartWatch(WeatherObserver):
    def update(self, temperature: float) -> None:
        print(f"Smart Watch: Current temperature: {temperature}")
```

```python
# Client code
weather_station = WeatherStation()

phone_display = PhoneDisplay()
smart_watch = SmartWatch()

weather_station.attach(phone_display)
weather_station.attach(smart_watch)

weather_station.set_temperature(28.5)
# Weather Station: Temperature changed to 28.5
# Phone Display: Temperature is now 28.5
# Smart Watch: Current temperature: 28.5

weather_station.set_temperature(30.0)
# Weather Station: Temperature changed to 30.0
# Phone Display: Temperature is now 30.0
# Smart Watch: Current temperature: 30.0

weather_station.detach(smart_watch)
weather_station.set_temperature(25.0)
# Weather Station: Temperature changed to 25.0
# Phone Display: Temperature is now 25.0
```

Observer - Use Case

- User interface event handling
- Data binding in MVC frameworks
- Systems requiring real-time updates.

Choosing the Right Design Pattern: A Summary

- **Understand the Problem**: What are you trying to achieve? What are the key challenges?
- **Consider the Context**: What are the requirements? Performance, scalability, maintainability?
- **Pattern Intent**: Does the pattern's purpose align with your needs (object creation, structure, behaviour)?
- **Benefits vs. Drawbacks**: Weigh the advantages (reusability, flexibility, etc.) against the potential downsides (complexity, over-engineering).
- **Prioritize Simplicity**: Choose the simplest pattern that solves the problem effectively. Avoid "pattern overkill."
- **Iterate and Refactor**: Design is an iterative process. Be prepared to refactor and adjust your pattern choices as the system evolves.

Scenario Challenge 1: Centralized Logging

- Scenario: Your application needs to log events to a single file. Multiple parts of the application should be able to write to this log file concurrently, but you need to ensure that only one instance of the logger exists to prevent data corruption or conflicts.

- Question: Which design pattern is the most appropriate for this scenario? Why?

Scenario Challenge 2: Dynamic UI Themes

- Scenario: You are building a user interface library. You want to allow users to easily customize the appearance of UI elements (e.g., buttons, text fields) by applying different themes (e.g., light, dark, high contrast) at runtime without modifying the core UI element classes.

- Question: Which design pattern is the most appropriate for this scenario? Why?

Scenario Challenge 3: Data Source Abstraction

- Scenario: Your application needs to read data from different data sources (e.g., a database, a CSV file, an API). Each data source has its own API and format. You want to provide a consistent interface to your application so that it doesn't need to know about the specifics of each data source.

- Question: Which design pattern is the most appropriate for this scenario?

Scenario Challenge 4: The Game Character Factory

- Scenario: You are developing a game. You need to create various types of game characters (e.g., Warrior, Mage, Archer). The creation process for each character type is slightly different, and you want to avoid complex if/else or switch statements in your game logic.

- Question: What pattern would make this easier, and why? What benefits and drawbacks might you encounter?

Scenario Challenge 5: Real-Time Updates

- Scenario: In a stock trading application, multiple UI components (e.g., a chart, a table, a news feed) need to be updated whenever the stock price changes. You want to design the system so that the UI components are loosely coupled and can be easily added or removed without affecting other parts of the application.

- Question: Which design pattern is the most appropriate for this scenario? Why?

Scenario Challenge 6: Report Generation System

- Scenario: You are developing a report generation system. The system needs to create various types of reports (e.g., sales reports, performance reports, summary reports) with different combinations of sections (e.g., header, body, footer, charts, tables). The order and specific content of these sections vary depending on the report type. You want to design the system to be flexible and easily extensible to support new report types in the future without modifying the core report generation logic.

- Question: Which design pattern is the most appropriate for this scenario? Why?

Key Takeaways

- Design patterns are reusable solutions to common design problems.
- They improve code quality, readability, and maintainability.
- Understanding the problem domain is crucial for choosing the right pattern.
- Avoid over-engineering by using patterns judiciously.

Q & A

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Lab